DOCUMENT 09 — RECIPE MATCHING & RECOMMENDATION ALGORITHM SPECIFICATION

1. Dokümanın Amacı

Smart Chef & Pantry AI uygulamasında kullanıcı tarafından seçilen malzemelere göre uygun tariflerin belirlenmesi, tariflerin eşleşme oranlarının hesaplanması, eksik malzemelerin tespit edilmesi ve tariflerin sıralanması bu algoritmanın temel sorumluluğudur.

Recommendation sistemi, uygulamanın temel iş mantıklarından biridir. Sistem, kullanıcıdan alınan malzeme listesini tariflerin malzeme listeleriyle karşılaştırarak hangi tariflerin mevcut malzemelerle hazırlanabileceğini belirler.

MVP aşamasında recommendation sistemi deterministik ve açıklanabilir kurallar kullanacaktır. Makine öğrenmesi veya LLM tabanlı bir sistem, tarif seçiminin temel karar mekanizması olarak kullanılmayacaktır.

Temel akış:

User Ingredients
↓
Ingredient Normalizer
↓
Normalized Ingredients
↓
Recipe Matcher
↓
Matched / Missing Ingredients
↓
Match Percentage
↓
Recommendation Engine
↓
Filtering & Ranking
↓
Recommended Recipes

Bu yapı, ilerleyen sürümlerde ML, NLP veya LLM tabanlı özelliklerin eklenmesine uygun olacak şekilde tasarlanacaktır.

---

2. Recommendation Sisteminin Girdileri

Recommendation sistemi iki temel veri grubuyla çalışır:

2.1. Kullanıcı Malzemeleri

Kullanıcı uygulamada sahip olduğu malzemeleri seçer.

Örneğin:

Domates
Yumurta
Peynir
Soğan

Frontend bu malzemeleri ingredient ID'leri olarak backend'e gönderir.

Örnek:

```
{
  "ingredient_ids": [
    "uuid-1",
    "uuid-2",
    "uuid-3",
    "uuid-4"
  ]
}
```

2.2. Tarif Malzemeleri

Her tarifin:

- Required ingredients
- Optional ingredients
- Quantity
- Unit
- Ingredient ID

bilgileri database üzerinden alınır.

Tarif malzemeleri `recipe_ingredients` tablosunda tutulur.

---

3. Ingredient Normalization

Malzemelerin doğrudan metin karşılaştırmasıyla eşleştirilmesi güvenilir değildir.

Örneğin:

Domates
domates
DOMATES
 domates

ifadelerinin aynı malzemeyi temsil ettiği kabul edilmelidir.

Bu nedenle sistemde `IngredientNormalizer` adı verilen bir bileşen kullanılacaktır.

Normalization işleminin temel amacı:

- küçük/büyük harf farklılıklarını kaldırmak
- baştaki ve sondaki boşlukları temizlemek
- gereksiz whitespace karakterlerini normalize etmek
- yazım biçimini standartlaştırmak
- malzeme adının canonical formuna ulaşmak

Örneğin:

" DOMATES "

şuna dönüştürülür:

"domates"

Ancak normalization işlemi malzemenin anlamını değiştirmemelidir.

Aşağıdaki malzemeler otomatik olarak aynı kabul edilmemelidir:

domates
çeri domates
domates sosu
kurutulmuş domates

Bunun nedeni, bu malzemelerin tariflerde birbirlerinin tamamen yerine geçebileceğinin garanti edilememesidir.

Bu nedenle normalization:

> Yazım biçimini standartlaştırmalı, malzemenin anlamsal türünü keyfi olarak değiştirmemelidir.

---

4. Ingredient Alias ve Variant Yönetimi

Bazı malzemelerin farklı isimlerle ifade edilmesi mümkündür.

Örneğin:

zeytinyağı
olive oil

gibi ifadeler kontrollü bir alias sistemiyle aynı canonical ingredient'a bağlanabilir.

İlerleyen aşamada aşağıdaki gibi ayrı bir yapı kullanılabilir:

ingredient_aliases

Önerilen alanlar:

id
canonical_ingredient_id
alias

Örneğin:

canonical_ingredient_id: uuid-123
alias: olive oil

şeklinde tutulabilir.

Böylece:

"olive oil"
↓
Ingredient Alias
↓
Canonical Ingredient
↓
"zeytinyağı"

şeklinde kontrollü eşleştirme yapılabilir.

Sistem otomatik olarak her benzer kelimeyi aynı malzeme olarak kabul etmemelidir.

Örneğin:

domates → domates sosu

şeklindeki bir dönüşüm otomatik olarak yapılmamalıdır.

---

5. Required ve Optional Ingredients

Tarif malzemeleri iki temel kategoriye ayrılacaktır.

5.1. Required Ingredients

Tarifin temel olarak ihtiyaç duyduğu malzemelerdir.

Örneğin:

Un
Yumurta
Süt
Tuz

Bu malzemeler match percentage hesaplamasına dahil edilir.

5.2. Optional Ingredients

Tarifi tamamlayan ancak tarifin temel eşleşme oranını düşürmemesi gereken malzemelerdir.

Örneğin:

Maydanoz
Karabiber
Pul biber

Database üzerinde bu ayrım:

recipe_ingredients.is_optional

alanı üzerinden belirlenir.

---

6. Match Percentage Calculation

MVP recommendation sisteminin temel metriği `match_percentage` olacaktır.

Temel formül:

Match Percentage =
Matched Required Ingredients
/
Total Required Ingredients
× 100

Örneğin bir tarifin required malzemeleri:

Un
Domates
Peynir
Zeytin

Kullanıcının sahip olduğu malzemeler:

Un
Domates
Peynir

ise:

Matched Required Ingredients = 3
Total Required Ingredients = 4

Sonuç:

3 / 4 × 100 = 75%

API'de:

```
{
  "match_percentage": 75.0
}
```

şeklinde döndürülebilir.

Flutter tarafında bu değer:

%75 Eşleşme

şeklinde gösterilebilir.

---

7. Optional Ingredients Match Calculation

Optional ingredients temel match percentage hesabına dahil edilmeyecektir.

Örneğin:

Required:
- Un
- Yumurta
- Süt

Optional:
- Maydanoz
- Karabiber

Kullanıcının sahip oldukları:

Un
Yumurta
Süt

ise:

3 / 3 × 100 = %100

olacaktır.

Kullanıcının maydanoz veya karabibere sahip olmaması match percentage değerini düşürmeyecektir.

Bunun nedeni, optional malzemelerin tarifin temel gereksinimleri olarak değerlendirilmemesidir.

---

8. Matched Ingredients ve Missing Ingredients

Recipe Matcher yalnızca match percentage hesaplamayacaktır.

Aynı zamanda:

matched_ingredients
missing_ingredients

listelerini oluşturacaktır.

Örneğin:

Recipe:
Menemen

Required:
- Domates
- Yumurta
- Soğan

User:
- Domates
- Yumurta

sonucunda:

Matched:
- Domates
- Yumurta

Missing:
- Soğan

oluşturulur.

API response örneği:


```
{
  "match_percentage": 66.67,
  "matched_ingredients": [
    {
      "ingredient_id": "uuid-1",
      "name": "Domates"
    },
    {
      "ingredient_id": "uuid-2",
      "name": "Yumurta"
    }
  ],
  "missing_ingredients": [
    {
      "ingredient_id": "uuid-3",
      "name": "Soğan"
```

```
    }
  ]
}
```

Bu yapı Flutter tarafında doğrudan kullanılabilir.

---

9. Recipe Filtering

Database'de çok sayıda tarif bulunabileceğinden tüm tariflerin kullanıcıya gösterilmesi gerekli değildir.

Örneğin sistemde:

500 recipes

bulunuyorsa ve kullanıcı:

Yumurta
Domates
Peynir

seçmişse her tarif için eşleşme hesaplanır.

Örneğin:

Recipe	Match Percentage
Menemen	%100
Omlet	%100
Domatesli Yumurta	%100
Peynirli Makarna	%50
Tavuk Çorbası	%0

MVP'de minimum eşleşme yüzdesi kullanılabilir.

Örneğin:

MIN_MATCH_PERCENTAGE = 30

olması durumunda:

match_percentage >= 30

olan tarifler recommendation sonucuna dahil edilir.

Bu değer kod içerisinde doğrudan sabit bir koşul olarak yazılmamalı, configuration üzerinden yönetilebilmelidir.

---

10. Recipe Ranking

Filtreleme sonrasında tarifler belirli kriterlere göre sıralanacaktır.

MVP'deki temel sıralama:

1. Match Percentage DESC
2. Missing Required Ingredients ASC
3. Matched Optional Ingredients DESC

Örneğin:

Recipe A
Match: %90
Missing: 1

Recipe B
Match: %90
Missing: 2

Recipe C
Match: %75
Missing: 0

sıralaması:

Recipe A
Recipe B
Recipe C

olacaktır.

Buradaki sıralama yalnızca önceden tanımlanmış teknik kriterlere dayanacaktır.

Sistem kullanıcıya subjektif olarak "en iyi tarif" gibi bir değerlendirme sunmayacaktır.

---

11. Duplicate Ingredient Handling

Kullanıcı aynı malzemeyi birden fazla kez seçerse malzeme eşleşmesi birden fazla kez sayılmamalıdır.

Örneğin:

Domates
Domates
Domates

şeklindeki input:

Domates

olarak değerlendirilmelidir.

Bu nedenle recommendation işleminden önce kullanıcı ingredient ID'leri unique hale getirilebilir.

Örneğin:

unique_ingredient_ids = set(user_ingredient_ids)

Burada quantity bilgisi farklı bir konudur.

Örneğin:

3 adet domates

ile:

1 adet domates

arasında miktar farkı vardır.

MVP recommendation sistemi öncelikli olarak:

Ingredient exists?

sorusuna cevap verecektir.

Miktar yeterliliği ilerleyen sürümlerde ayrıca ele alınacaktır.

---

12. Quantity ve Unit Handling

Recipe ingredients içerisinde miktar ve birim bilgileri tutulacaktır.

Örneğin:

200 g un
2 adet yumurta
100 ml süt

Ancak MVP'deki temel matching algoritması miktarları zorunlu olarak karşılaştırmayacaktır.

İlk versiyonda:

Ingredient exists → Matched
Ingredient does not exist → Missing

mantığı kullanılacaktır.

İlerleyen sürümde quantity-aware matching eklenebilir.

Örneğin:

Recipe requires:
200 g flour

User has:
500 g flour

Result:
Sufficient

ve:

Recipe requires:
500 g flour

User has:
200 g flour

Result:
Insufficient

şeklinde daha gelişmiş bir kontrol uygulanabilir.

Bu özellik MVP kapsamı dışında tutulacaktır.

---

13. Edge Cases

13.1. Empty Ingredient List

Kullanıcı boş ingredient listesi gönderirse:

```
{
  "ingredient_ids": []
}
```

API:

EMPTY_INGREDIENT_LIST

hatası döndürmelidir.

13.2. Invalid Ingredient ID

Gönderilen ingredient ID database'de bulunmuyorsa:

INGREDIENT_NOT_FOUND

hatası döndürülmelidir.

13.3. Recipe Without Required Ingredients

Bir tarifin required ingredient sayısı:

0

olmamalıdır.

Çünkü:

0 / 0

şeklinde bir match percentage hesaplaması yapılamaz.

Bu nedenle recipe oluşturma/güncelleme sırasında en az bir required ingredient bulunması önerilir.

13.4. No Matching Recipes

Hiçbir tarif minimum eşleşme oranını karşılamıyorsa sistem bunu server error olarak değerlendirmemelidir.

Örneğin:

```
{
  "success": true,
  "data": {
    "recipes": []
  },
  "message": "No matching recipes found."
}
```

döndürülebilir.

13.5. Inactive Recipes

Database'de pasif durumda olan tarifler recommendation sonuçlarına dahil edilmemelidir.

İleride recipe tablosunda:

is_active

alanı bulunabilir.

is_active = false

olan tarifler sistem tarafından filtrelenmelidir.

---

14. Example Recommendation Scenario

Kullanıcının seçtiği malzemeler:

Domates
Yumurta

Peynir

Tarif:

Menemen

Required ingredients:

Domates
Yumurta
Soğan

Optional ingredients:

Maydanoz

Kullanıcının sahip oldukları:

Domates ✓
Yumurta ✓
Peynir ✓

Eşleşme:

Matched Required:
- Domates
- Yumurta

Missing Required:
- Soğan

Required ingredient sayısı:

3

Matched required ingredient sayısı:

2

Match percentage:

2 / 3 × 100 = 66.67%

Sonuç:

Match Percentage: %66.67

Flutter tarafında yuvarlanarak:

%67 Eşleşme

şeklinde gösterilebilir.

---

15. Algorithm Pseudocode

Recommendation algoritmasının temel mantığı:

```
FUNCTION recommend_recipes(user_ingredient_ids):

    IF user_ingredient_ids is empty:
        RETURN EMPTY_INGREDIENT_LIST

    normalized_user_ingredients =
        IngredientNormalizer.normalize(user_ingredient_ids)

    recipes =
        RecipeRepository.get_active_recipes()

    recommendations = []

    FOR each recipe IN recipes:

        required_ingredients =
            recipe.required_ingredients

        optional_ingredients =
            recipe.optional_ingredients

        matched_required = []
        missing_required = []

        FOR each ingredient IN required_ingredients:

            IF ingredient.id IN normalized_user_ingredients:
```

```
            matched_required.append(ingredient)

        ELSE:

            missing_required.append(ingredient)

    total_required =
        length(required_ingredients)

    matched_count =
        length(matched_required)

    match_percentage =
        (matched_count / total_required) * 100

    matched_optional = []

    FOR each ingredient IN optional_ingredients:

        IF ingredient.id IN normalized_user_ingredients:

            matched_optional.append(ingredient)

    IF match_percentage >= MIN_MATCH_PERCENTAGE:

        recommendations.append(
            recipe,
            match_percentage,
            matched_required,
            missing_required,
            matched_optional
        )

SORT recommendations BY:

    match_percentage DESC
    missing_required_count ASC
    matched_optional_count DESC

RETURN recommendations
```

---

16. Backend Component Responsibilities

Recommendation sistemi tek bir büyük fonksiyon halinde oluşturulmayacaktır.

Sorumluluklar aşağıdaki bileşenlere ayrılacaktır.

16.1. IngredientNormalizer

Dosya:

app/ai/ingredient_normalizer.py

Sorumlulukları:

- ingredient normalization
- lowercase conversion
- whitespace normalization
- canonical ingredient kontrolü
- alias desteği

Örnek:

```
class IngredientNormalizer:

    def normalize(self, ingredient):
        ...
```

16.2. RecipeMatcher

Dosya:

app/ai/recipe_matcher.py

Sorumlulukları:

- required ingredient matching
- optional ingredient matching
- missing ingredient detection
- matched ingredient detection
- match percentage calculation

Örnek:

```
class RecipeMatcher:

    def match(self, recipe, user_ingredients):
        ...
```

Çıktı:

matched_ingredients
missing_ingredients
matched_optional_ingredients
match_percentage

16.3. RecommendationEngine

Dosya:

app/ai/recommendation_engine.py

Sorumlulukları:

- tarifleri almak
- RecipeMatcher kullanmak
- minimum eşleşme oranını uygulamak
- tarifleri sıralamak
- recommendation sonucunu oluşturmak

Örnek:

```
class RecommendationEngine:

    def recommend(self, recipes, user_ingredients):
        ...
```

Temel akış:

```
Recipes
  ↓
RecipeMatcher
  ↓
Match Results
  ↓
Filter
  ↓
Ranking
  ↓
Recommendations
```

---

17. RecipeService ile İlişkisi

`RecipeService`, recommendation işlemini yönetecek ancak matching algoritmasının bütün detaylarını kendisi gerçekleştirmeyecektir.

Backend akışı:

POST /api/recipes/recommend
↓
Recipe Route
↓
Request Schema Validation
↓
Recipe Service
↓
Recommendation Engine
↓
Recipe Matcher
↓
Ingredient Normalizer
↓
Repository
↓
PostgreSQL

Bu yapı sayesinde recommendation algoritması API katmanından bağımsız tutulur.

---

18. API Response

Endpoint:

POST /api/recipes/recommend

Request:

```
{
  "ingredient_ids": [
    "uuid-tomato",
    "uuid-egg",
    "uuid-cheese"
  ]
}
```

Response:

```
{
  "success": true,
  "data": {
    "recipes": [
      {
        "id": "recipe-001",
        "title": "Menemen",
        "image_url": "https://example.com/image.jpg",
        "match_percentage": 66.67,
        "matched_ingredients": [
          {
            "ingredient_id": "uuid-tomato",
            "name": "Domates"
          },
          {
            "ingredient_id": "uuid-egg",
            "name": "Yumurta"
          }
        ],
        "missing_ingredients": [
          {
            "ingredient_id": "uuid-onion",
            "name": "Soğan"
          }
        ]
      }
    ]
  }
}
```


Flutter tarafında match percentage yeniden hesaplanmayacaktır.

Backend:

66.67

değerini üretir.

Flutter:

%67 Eşleşme

olarak gösterir.

Bu sayede business logic backend'de merkezi olarak tutulur.

---

19. MVP Recommendation Strategy

MVP'de kullanılacak sistem:

Deterministic Ingredient Matching
+
Required / Optional Ingredients
+
Match Percentage
+
Missing Ingredients
+
Minimum Match Threshold
+
Recipe Ranking

olacaktır.

MVP'de aşağıdaki yöntemler kullanılmayacaktır:

Real-time market price optimization
Advanced personalization
Collaborative filtering
LLM-based recipe selection
Complex machine learning recommendation
User-to-user recommendation

Bu özellikler ilerleyen versiyonlarda değerlendirilebilir.

---

20. Future Weighted Recommendation

MVP sonrasında recommendation sistemine ağırlıklı skor sistemi eklenebilir.

Örneğin ileride:

Required Ingredient Match 70%
Optional Ingredient Match 10%
Ingredient Preference 10%
Recipe Category Compatibility 10%

gibi bir model değerlendirilebilir.

Ancak bu ağırlıklar MVP'de kullanılmayacaktır.

Ağırlıkların gerçek kullanıcı davranışları ve test sonuçları incelendikten sonra belirlenmesi daha doğru olacaktır.

---

21. Future Machine Learning Integration

İlerleyen sürümlerde kullanıcı davranışlarından recommendation sinyalleri üretilebilir.

Örneğin:

Recipe Viewed
Recipe Favorited
Recipe Added to Cart
Recipe Source Opened
Ingredient Frequently Selected

gibi davranışlar kayıt altına alınabilir.

Bu verilerden kullanıcı tercihleri çıkarılabilir.

Örneğin:

User
↓
Frequently views vegetarian recipes
↓
Frequently favorites Italian recipes
↓
Frequently selects chicken

gibi davranış profilleri oluşturulabilir.

Daha sonra:

Content-Based Recommendation

veya daha gelişmiş:

Learning-to-Rank

yaklaşımları değerlendirilebilir.

Mevcut recommendation mimarisi korunarak bu sistemlerin eklenebilmesi hedeflenmektedir.

---

22. Future NLP / LLM Integration

İlerleyen aşamada kullanıcıların doğal dil kullanarak malzeme girmesine izin verilebilir.

Örneğin kullanıcı:

Evde iki tane domates, yumurta ve biraz peynir var.

şeklinde bir giriş yapabilir.

NLP/LLM katmanı bu ifadeyi:

```
{
  "ingredients": [
    "domates",
    "yumurta",
    "peynir"
  ]
}
```

şeklinde yapılandırılmış veriye dönüştürebilir.

Ardından:

Natural Language Input
↓
NLP / LLM
↓
Ingredient Extraction
↓
Ingredient IDs
↓
Recipe Matcher
↓
Recommendation Engine

akışı kullanılabilir.

Buradaki kritik mimari karar:

> LLM tarif seçiminin ana karar mekanizması olmayacaktır.

LLM'nin görevi doğal dili yapılandırılmış ingredient input'a dönüştürmek olabilir.

Tarif eşleştirme ve ranking işlemleri mevcut backend algoritması tarafından gerçekleştirilecektir.

Bu yaklaşım:

- daha öngörülebilir
- daha test edilebilir
- daha kolay debug edilebilir
- daha kontrollü
- API sözleşmesine daha uygun

bir yapı sağlar.

---

23. Testing Strategy

Recommendation algoritması için aşağıdaki test senaryoları oluşturulacaktır:

Test Senaryosu	Beklenen Sonuç
Tüm required ingredients mevcut	%100
Required ingredients'ın yarısı mevcut	%50
Hiçbir required ingredient mevcut değil	%0
Optional ingredient eksik	Match percentage değişmez
Duplicate ingredient	Bir kez sayılır
Boş ingredient listesi	Validation error
Geçersiz ingredient ID	Ingredient not found
Aynı match percentage	Missing ingredient sayısına göre sıralama
Required ingredient sayısı 0	Validation/data error
Pasif tarif	Sonuçlara dahil edilmez
Geçerli alias	Canonical ingredient ile eşleşir
Tanımsız benzer ingredient	Otomatik eşleştirilmez
Minimum threshold altında	Sonuçlara dahil edilmez

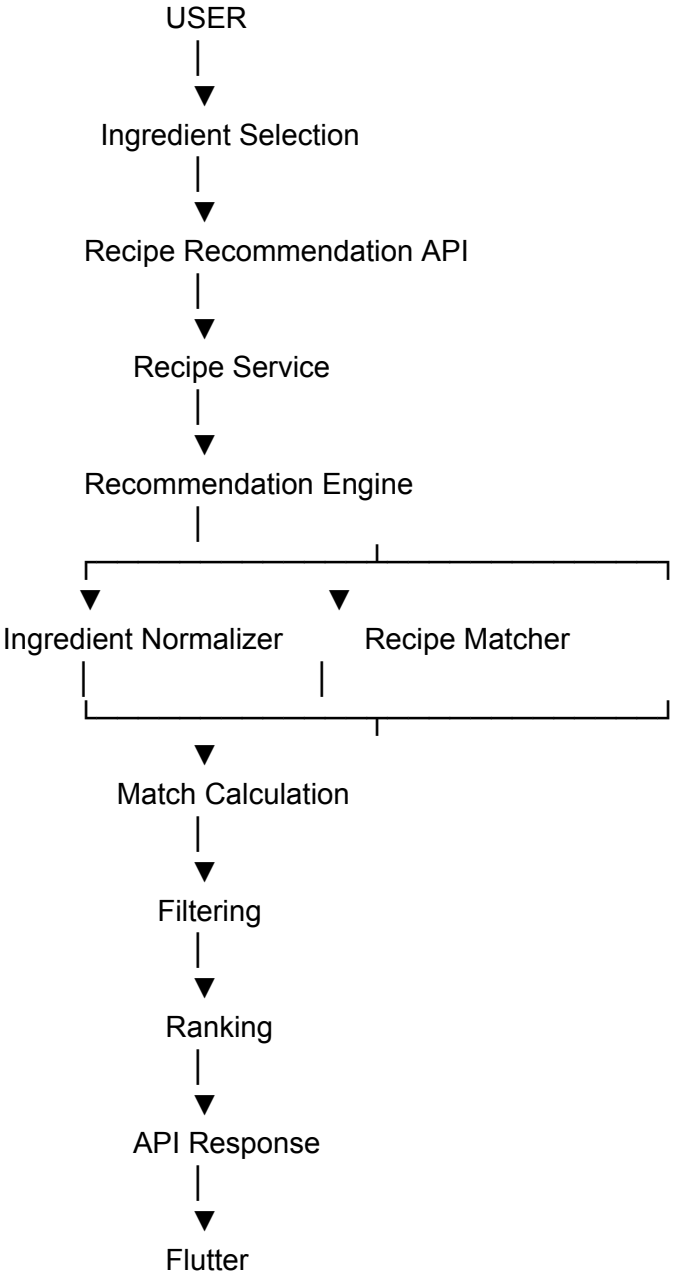
Özellikle aşağıdaki üç konu unit testlerde kritik olacaktır:

1. Match Percentage
2. Missing Ingredient Detection
3. Recipe Ranking

---

24. Final Architecture

Smart Chef & Pantry AI recommendation mimarisi:



MVP

Deterministic Matching
+
Required / Optional Ingredients
+
Match Percentage
+
Missing Ingredients
+
Recipe Ranking

Future V2

Weighted Scoring
+
Content-Based Recommendation

Future V3

User Behavior
+
Machine Learning Recommendation

NLP / LLM Extension

Natural Language Input
↓
NLP / LLM
↓
Ingredient Extraction
↓
Existing Recommendation Engine

Bu mimari ile recommendation sistemi ilk aşamada basit, kontrol edilebilir ve test edilebilir tutulurken ilerleyen aşamalarda yapay zekâ tabanlı özelliklerin mevcut sistemi bozmadan eklenebilmesi sağlanacaktır.